

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array.

The `Player` class is provided for you in your editor. It has **2** fields: a ***name*** String and a ***score*** integer.

Given an array of ***n*** `Player` objects, write a comparator that sorts them in order of decreasing score; if **2** or more players have the same score, sort those players alphabetically by name. To do this, you must create a `Checker` class that implements the `Comparator` interface, then write an `int compare(Player a, Player b)` method implementing the `Comparator.compare(T o1, T o2)` method.

Input Format

Input from `stdin` is handled by the locked stub code in the `Solution` class.

The first line contains an integer, ***n***, denoting the number of players.

Each of the ***n*** subsequent lines contains a player's ***name*** and ***score***, respectively.

Constraints

- $0 \leq score \leq 1000$
- **2** players can have the same name.
- Player names consist of lowercase English letters.

Output Format

You are not responsible for printing any output to `stdout`. The locked stub code in `Solution` will create a `Checker` object, use it to sort the `Player` array, and print each sorted element.

Sample Input

```
5
amy 100
david 100
heraldo 50
aakashsha 75
aleksa 150
```

Sample Output

```
aleksa 150
```

55
56
57
58

`sc.close();`
`}`
`}`

Line: 58 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations

You solved this challenge. Would you like to challenge your friends?

[f](#)[t](#)[in](#)

Next Challenge

Test case 0

Test case 1

Test case 2

Test case 3

Test case 4

Test case 5

Compiler Message

Success

Input (stdin)

1 5
2 amy 100
3 david 100
4 heraldo 50
5 aakashsha 75
6 aleksa 150

Expected Output

Download

Download

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array. The `Player` class is provided in the editor below. It has two fields:

- 1. `name`: a string.
- 2. `score`: an integer.

Given an array of `n` `Player` objects, write a comparator that sorts them in order of decreasing score. If `2` or more players have the same score, sort those players alphabetically ascending by name. To do this, you must create a `Checker` class that implements the `Comparator` interface, then write an `int compare(Player a, Player b)` method implementing the `Comparator.compare(T o1, T o2)` method. In short, when sorting in ascending order, a comparator function returns `-1` if `a < b`, `0` if `a = b`, and `1` if `a > b`.

Declare a `Checker` class that implements the comparator method as described. It should sort first descending by score, then ascending by name. The code stub reads the input, creates a list of `Player` objects, uses your method to sort the data, and prints it out properly.

Example

`n = 3 data = [[Smith, 20], [Jones, 15], [Jones, 20]]`

Sort the list as `datasorted = [[Jones, 20], [Smith, 20], [Jones, 15]]`. Sort first descending by score, then ascending by name.

Input Format

The first line contains an integer, `n`, the number of players.

Each of the next `n` lines contains a player's `name` and `score`, a string and an integer.

Constraints

- $0 \leq score \leq 1000$
- Two or more players can have the same name.
- Player names consist of lowercase English alphabetic letters.

Output Format

You are not responsible for printing any output to stdout. Locked stub code in Solution will instantiate a `Checker` object, use it to sort the `Player` array, and print each sorted element.

Sample Input

```
40 | | | }
41 | | }
42 | }
43 |
```

Line: 43 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations

You solved this challenge. Would you like to challenge your friends?

[f](#)[t](#)[in](#)

Next Challenge

Test case 0

Test case 1

Test case 2

Test case 3

Test case 4

Test case 5

Test case 6

Compiler Message

Success

Input (stdin)

```
1 5
2 amy 100
3 david 100
4 heraldo 50
5 aakansha 75
6 aleksa 150
```

Expected Output

Download Code

Download Code

You are given a list of student information: ID, FirstName, and CGPA. Your task is to rearrange them according to their CGPA in decreasing order. If two student have the same CGPA, then arrange them according to their first name in alphabetical order. If those two students also have the same first name, then order them according to their ID. No two students have the same ID.

Hint: You can use comparators to sort a list of objects. See the [oracle docs](#) to learn about comparators.

Input Format

The first line of input contains an integer *N*, representing the total number of students. The next *N* lines contains a list of student information in the following structure:

```
ID Name CGPA
```

Constraints

- $2 \leq N \leq 1000$
- $0 \leq ID \leq 100000$
- $5 \leq |Name| \leq 30$
- $0 \leq CGPA \leq 4.00$

The name contains only lowercase English letters. The *ID* contains only integer numbers without leading zeros. The CGPA will contain, at most, 2 digits after the decimal point.

Output Format

After rearranging the students according to the above rules, print the first name of each student on a separate line.

Sample Input

```
5
33 Rumpa 3.68
85 Ashis 3.85
56 Samiha 3.75
19 Samara 3.75
22 Fahim 3.76
```

Sample Output

Upload Code as File☐ Test against custom input

Run CodeSubmit Code

You have earned 10.00 points!
You are now 30 points away from the 3rd star for your java badge.

0%50/80

Congratulations

You solved this challenge. Would you like to challenge your friends?

Next Challenge

Test case 0Test case 1Test case 2Test case 3Test case 4Test case 5Test case 6

Compiler Message

Success

Input (stdin)

1 5
2 33 Rumpa 3.68
3 85 Ashis 3.85
4 56 Samiha 3.75
5 19 Samara 3.75
6 22 Fahim 3.76

Expected Output

DownloadDownload

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array.

The Player class is provided for you in your editor. It has **2** fields: a **name** String and a **score** integer.

Given an array of **n** Player objects, write a comparator that sorts them in order of decreasing score; if **2** or more players have the same score, sort those players alphabetically by name. To do this, you must create a Checker class that implements the Comparator interface, then write an int compare(Player a, Player b) method implementing the `Comparator.compare(T o1, T o2)` method.

Input Format

Input from stdin is handled by the locked stub code in the Solution class.

The first line contains an integer, **n**, denoting the number of players.

Each of the **n** subsequent lines contains a player's **name** and **score**, respectively.

Constraints

- $0 \leq \text{score} \leq 1000$
- **2** players can have the same name.
- Player names consist of lowercase English letters.

Output Format

You are not responsible for printing any output to stdout. The locked stub code in Solution will create a Checker object, use it to sort the Player array, and print each sorted element.

Sample Input

```
5
amy 100
david 100
heraldo 50
aakashsha 75
aleksa 150
```

Sample Output

```
aleksa 150
amy 100
```

```
45 Checker checker = new Checker();
46
47 // Sort the players
48 Arrays.sort(players, checker);
49
50 // Print sorted players
51 for (Player p : players) {
52     System.out.println(p.name + " " + p.score);
53 }
54
55 sc.close();
56 }
57
58
```

Line: 58 Co

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Test case 0

Compiler Message

Test case 1

Success

Test case 2

Input (stdin)

Test case 3

Test case 4

Test case 5

```
1 5
2 amy 100
3 david 100
4 heraldo 50
5 aakashsha 75
6 aleksa 150
```

Expected Output

Download

Download

This Java 8 challenge tests your knowledge of [Lambda expressions](#)!

Write the following methods that return a lambda expression performing a specified action:

1. PerformOperation isOdd(): The lambda expression must return *true* if a number is odd or *false* if it is even.
2. PerformOperation isPrime(): The lambda expression must return *true* if a number is prime or *false* if it is composite.
3. PerformOperation isPalindrome(): The lambda expression must return *true* if a number is a palindrome or *false* if it is not.

Input Format

Input is handled for you by the locked stub code in your editor.

Output Format

The locked stub code in your editor will print *T* lines of output.

Sample Input

The first line contains an integer, *T* (the number of test cases).

The *T* subsequent lines each describe a test case in the form of **2** space-separated integers:

The first integer specifies the condition to check for (**1** for Odd/Even, **2** for Prime, or **3** for Palindrome). The second integer denotes the number to be checked.

```
5
1 4
2 5
3 898
1 3
2 12
```

Sample Output

```
EVEN
PRIME
PALINDROME
ODD
COMPOSITE
```

Upload Code as File

☐ Test against custom input

Run Code

Submit Code



You have earned 30.00 points!
You are now 20 points away from the 2nd star for your java badge.

20%

30/50

Congratulations

You solved this challenge. Would you like to challenge your friends? [f](#) [t](#) [in](#)

Next Challenge

Test case 0

Test case 1

Test case 2

Compiler Message

Success

Input (stdin)

1	5
2	1 4
3	2 5
4	3 898
5	1 3
6	2 12

Expected Output

Download

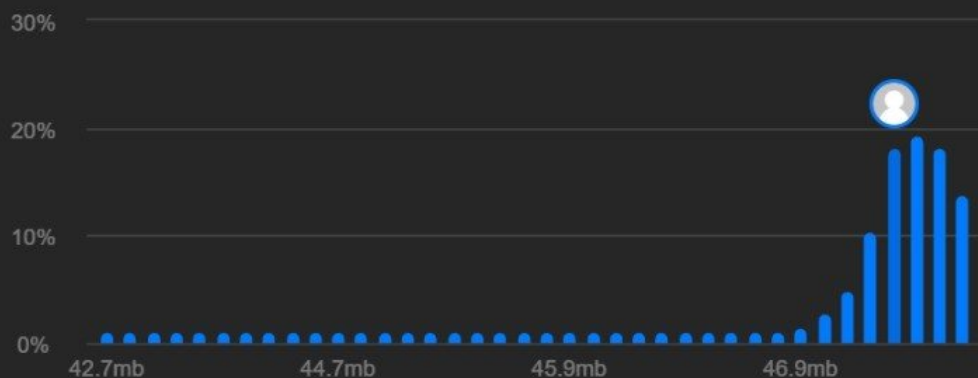
Download

Description Accepted Editorial Solutions Submissions

All Submissions

Memory

47.33 MB | Beats 60.43%



42.7mb 44.7mb 45.9mb 46.9mb

Code | Java

Code

Java

Auto

```
26 System.out.println(sol.pivotIndex(nums2)); // -1
27 System.out.println(sol.pivotIndex(nums3)); // 0
28 }
29 }
30
```

Saved

Ln 30, Co

Testcase Test Result

Accepted Runtime: 0 ms

Case 1

Case 2

Case 3

Input

nums =

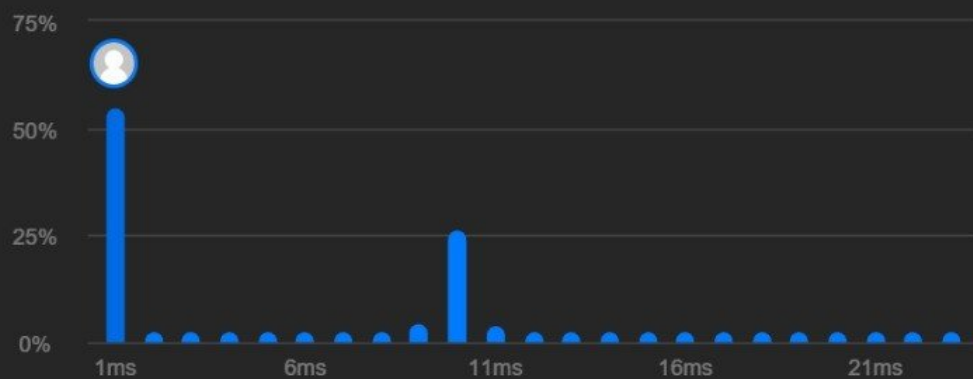
[1,7,3,6,5,6]

Description | Accepted × | Editorial | Solutions | Submissior <

All Submissions

Memory

46.93 MB | Beats 94.79% 🏆



Code | Java

</> Code

Java ▾ 🔒 Auto

```
31      System.out.println(java.util.Arrays.toString(sol.  
32          sortedSquares(nums2))); // [4, 9, 9, 49, 121]  
33  }  
34
```

Saved

Ln 34, Col

☑ Testcase | >_ Test Result

Accepted Runtime: 0 ms

☑ Case 1

☑ Case 2

Input

nums =
[-4, -1, 0, 3, 10]

All Submissions

🕒 Runtime

1 ms | Beats 16.06%

⚙️ Memory

44.66 MB | Beats 37.52%

100%

50%

0%

1ms

2ms

3ms

4ms

Runtime: 3ms

Java

```
class Solution {  
    public int maximumWealth(int[][] accounts) {  
        ArrayList<Integer>a=new ArrayList<>();  
  
        for(int i=0;i<accounts.length;i++)
```

☑️ Testcase | >_ Test Result

Accepted Runtime: 0 ms

☑️ Case 1

☑️ Case 2

☑️ Case 3

Input

accounts =
[[1,2,3],[3,2,1]]

All Submissions

0 ms | Beats 100.00%

Memory

44.52 MB | Beats 17.36%



Code

Java Auto

```
10
11 // Example usage
12 public static void main(String[] args) {
13     Solution sol = new Solution();
14     int[] nums1 = {1, 2, 3, 4};
15     int[] nums2 = {1, 1, 1, 1, 1};
16 }
```

Saved

Testcase | Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =
[1,2,3,4]